

tf.compat.v1.data.Iterator Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/data/Iterator

Represents the state of iterating through a Dataset.

Function Signatures

```
tf.compat.v1.data.Iterator( iterator_resource, initializer,
output_types, output_shapes, output_classes )
```

Code Examples

```
tf.compat.v1.data.Iterator(
    iterator_resource, initializer, output_types, output_shapes,
    output_classes
)
```

```
tf.compat.v1.data.Iterator(
    iterator_resource, initializer, output_types, output_shapes,
    output_classes
)
```

```
@staticmethod
from_string_handle(
    string_handle, output_types, output_shapes=None,
    output_classes=None
)
```

```
@staticmethod
```

```
from_string_handle(
    string_handle, output_types, output_shapes=None,
    output_classes=None
)
```

```
train_iterator = tf.data.Dataset(...).make_one_shot_iterator()
train_iterator_handle = sess.run(train_iterator.string_handle())
test_iterator = tf.data.Dataset(...).make_one_shot_iterator()
test_iterator_handle = sess.run(test_iterator.string_handle())
handle = tf.compat.v1.placeholder(tf.string, shape=[])
iterator = tf.data.Iterator.from_string_handle(
    handle, train_iterator.output_types)
next_element = iterator.get_next()
loss = f(next_element)
train_loss = sess.run(loss, feed_dict={handle:
    train_iterator_handle})
test_loss = sess.run(loss, feed_dict={handle:
    test_iterator_handle})
```

```
train_iterator = tf.data.Dataset(...).make_one_shot_iterator()
train_iterator_handle = sess.run(train_iterator.string_handle())
test_iterator = tf.data.Dataset(...).make_one_shot_iterator()
test_iterator_handle = sess.run(test_iterator.string_handle())
handle = tf.compat.v1.placeholder(tf.string, shape=[])
iterator = tf.data.Iterator.from_string_handle(
    handle, train_iterator.output_types)
next_element = iterator.get_next()
loss = f(next_element)
train_loss = sess.run(loss, feed_dict={handle:
    train_iterator_handle})
test_loss = sess.run(loss, feed_dict={handle:
    test_iterator_handle})
```

```
@staticmethod
from_structure(
    output_types, output_shapes=None, shared_name=None,
    output_classes=None
)
```

Documentation

Represents the state of iterating through a Dataset.

Raises

Attributes

For more information,
read this guide.

The expected values are `tf.Tensor` and `tf.sparse.SparseTensor`.

Methods

`## from_string_handle`

[View source](#)

Creates a new, uninitialized Iterator based on the given handle.

This method allows you to define a "feedable" iterator where you can choose between concrete iterators by feeding a value in a `tf.Session.run` call.

In that case, `string_handle` would be a `tf.compat.v1.placeholder`, and you would

feed it with the value of `tf.data.Iterator.string_handle` in each step.

For example, if you had two iterators that marked the current position in a training dataset and a test dataset, you could choose which to use in each step as follows:

from_structure

[View source](#)

Creates a new, uninitialized Iterator with the given structure.

This iterator-constructing method can be used to create an iterator that is reusable with many different datasets.

The returned iterator is not bound to a particular dataset, and it has no initializer. To initialize the iterator, run the operation returned by `Iterator.make_initializer(dataset)`.

The following is an example

get_next

[View source](#)

Returns the next element.

In graph mode, you should typically call this method once and use its result as the input to another computation. A typical loop will then call `tf.Session.run` on the result of that computation. The loop will terminate when the `Iterator.get_next()` operation raises `tf.errors.OutOfRangeError`. The following skeleton shows how to use this method when building a training loop:

get_next_as_optional

[View source](#)

make_initializer

[View source](#)

Returns a `tf.Operation` that initializes this iterator on dataset.

string_handle

[View source](#)

Returns a string-valued `tf.Tensor` that represents this iterator.